CMPE3209 Computer Organization — Proje Teslim Tutanağı

Kullanım talimatı

Bu tutanak projenin tek resmi teslim belgesidir. Her kilometre taşında (M1, M2 ve final; tümü pazartesi) ilgili adım doldurulur ve aynı dosya güncellenerek yeniden teslim edilir. Ayrıntılı görev tanımları proje yönergesindedir; bu tutanak yalnızca ne istendiğini ve nereye yazılacağını gösterir.

- Tüm sayılar birimiyle yazılır (byte, cycle, ns, GHz). Hex değerler `0x` önekiyle yazılır.
- **Varsayım** olan her değer "(varsayım)" etiketiyle işaretlenir.
- Tablolara sığmayan çıktılar (assembly listesi, objdump, CSV izi, ekran görüntüleri) Git deposuna konur. Tutanakta dosya adıyla atıf yapılır.
- Boş bırakılan zorunlu alan, o adımın puanını sıfırlar.

Öğrenci bilgileri

Alan	Değer
Ad Soyad	Mehmet Batuhan Özer
Öğrenci numarası (SID)	5001230025
E-posta	`batuhanozer1042@gmail.com`
Seçilen görev (G1–G5)	G5 — Run-length sıkıştırma
Seçilen ISA ve sürümü (ör. RV32IM)	x86-64 (Microsoft x64 ABI, PE/COFF)
Derleyici ve sürümü	clang-cl, LLVM 21.1.8 (MSVC 14.44.35207)
Simülatör / araçlar	Visual Studio 2022 (debugger), `llvm-objdump` 21.1.8; CMake 3.31.6 + Ninja 1.12.1
Git deposu adresi	`https://github.com/MBatuhanOzer/Computer_Organization_Project`

Adım 0 — Kişiye özel parametreler

Bu adım M1 ile birlikte teslim edilir. Hatalı bir parametre, ona dayanan tüm adımları geçersiz kılar; öğretim üyesi bu tabloyu M1'de kontrol eder.

Sembol	Tanım	Hesap (adımlarıyla)	Sonuç
d	SID'in son hanesi	5001230025	**5**
D2	SID'in son iki hanesi	5001230025	**25**
S	SID'in hanelerinin toplamı	5+0+0+1+2+3+0+0+2+5	**18**
N	24 + (D2 mod 25)	24 + (25 mod 25) = 24 + 0	**24 eleman**
Cache	d'ye göre: boyut / line / mapping / line sayısı / set sayısı	d = 5 satırı	**256 byte / 16 byte / 4-way set-assoc. / 16 line / 4 set**

Sembol	Tanım	Hesap (adımlarıyla)	Sonuç
Write policy	d çift → write-back, d tek → write-through	d = 5 (tek)	**write-through**
k	(S mod 7) + 3	(18 mod 7) + 3 = 4 + 3	**7**
Interrupt türü	S çift → Timer, S tek → I/O	S = 18 (çift)	**Timer**
f	1.0 + 0.1 × d GHz (varsayım)	1.0 + 0.1 × 5	**1,5 GHz** (varsayım)
T1 / T2	T1 = 1 cycle; T2 = 20 + 5 × (d mod 4) cycle (varsayım)	T2 = 20 + 5 × 1	**1 cycle / 25 cycle** (varsayım)
Nc	2 + (S mod 7) (varsayım)	2 + 4	**6 çekirdek** (varsayım)
Görev parametresi	Seçilen göreve göre (T, P, K, yön veya R)	G5: R = 2 + (SID mod 3); 5001230025 mod 3 = 0	**R = 2**, B = 0

Türetilmiş: $\tau = 1/f = 1/(1{,}5\ \text{GHz}) = \mathbf{0{,}667}$ ns (varsayım). Tutarlılık: 256 byte / 16 byte = 16 line; 16 line / 4 yol = 4 set.

Adres alanları (ISA'nın adres genişliği s + w bit; 64-bit ISA'da düşük 32 bit kullanılır):

Alan	Bit sayısı	Hesap
Word (w)	**4 bit**	line boyu 16 byte $= 2^4$; adres[3:0]
Line (r) veya set (d)	**2 bit**	set sayısı $4 = 2^2$; adres[5:4]
Tag ($s - r$ veya $s - d$)	**26 bit**	$s + w = 32 \Rightarrow s = 28$; $28 - 2$; adres[31:6]
Toplam	**32 bit**	$26 + 2 + 4$

Adım 1 — Görev seçimi ve C programı (M1: 2 Kasım, Pazartesi)

1.1 Görev seçimi (yalnızca birini işaretleyin):

- ☐ G1 Biyomedikal sinyal alarmı
- ☐ G2 Sınav notu dağılımı
- ☐ G3 Mesaj şifreleme ve bütünlük
- ☐ G4 Sıralama ve medyan
- ☒ **G5 Run-length sıkıştırma**

1.2 Çözüm tasarımı (3–5 cümle: hangi algoritma, hangi fonksiyonlar, hangi veri yerleşimi ve neden):

Program üç global dizi üzerinde çalışır: `v[24]` LCG ile üretilen $\{0, 1\}$ sembollerini, `enc[24]` kodlanmış (değer, tekrar) çiftlerini, `dec[24]` ise geri açılmış diziyi tutar. `rle_encode(n)` `v` dizisini tek geçişte tarar; dış döngü her koşu için bir kez döner, iç döngü aynı sembolün tekrar sayısını sayar ve her koşu `(v[i] << 8) | run` biçiminde tek bir 32-bit word'e paketlenerek `enc` dizisine yazılır, böylece shift, OR ve dizi store'u aynı satırda görünür hale gelir. `rle_decode(len)` bu paketleri `>> 8` ve `& 0xFF` ile ayrıştırıp `dec` dizisine açar; her iki fonksiyon da parametre alıp değer döndürür. Doğrulama iki aşamalıdır: önce eleman sayısı (`out == N`), sonra üçüncü geçişte eleman değerleri karşılaştırılır. Veri yerleşimi bilinçli olarak üç ayrı global dizi şeklinde seçilmiştir: üç dizi 15 cache line'a karşılık gelir, 256 byte'lık cache ise 16 line tutar; bu,

4-way set'lerden tam birinin taşmasını sağlayarak Aşama 4'te hem spatial locality'nin hem de conflict miss'in aynı iz üzerinde gözlenmesine imkân verir.

1.3 C kaynak kodu. Satır numaralı kod aşağıdaki bloğa yapıştırılır (25–60 satır):

```c
// Name: Mehmet Batuhan Ozer
// Student Number: 5001230025
// Gorev G5: Run-Length Encoding (RLE)
// Personalized parameters: N = 24, R = 2, B = 0

#define N 24                    // 24 + (D2 mod 25) = 24 + (25 mod 25)
#define R 2                     // 2 + (SID mod 3) = 2 + 0
#define B 0                     // task definition

typedef unsigned int u32;       // 32-bit unsigned
typedef int s32;                // 32-bit signed

const int SID = 1230025;        // Y6: last 9 digits of 5001230025 (exceeds 2^31-1). Kept as const int exactly as the assignment specifies; s32 is a typedef of int anyway.

s32 v[N];                       // Y1: Original input array (global, 3 passes)
s32 enc[N];                     // RLE Encoded Array
s32 dec[N];                     // Decoded Array (Must be the same as the original array after decoding)

s32 enc_len = 0;                // Y7: number of runs produced
s32 ratio_x100 = 0;             // Y7: 100 * enc_len / N
s32 decode_ok = 0;              // Y7: 1 if dec[] matches v[]

// Y3: takes a parameter, returns a value. Encodes v[0..n-1] into enc[], returns the run count
s32 rle_encode(s32 n)
{
    s32 i = 0;
    s32 k = 0;
    while (i < n) {                         // Y1: 2nd pass over v
        s32 run = 1;
        while (i + run < n && v[i + run] == v[i]) { run++; }   // Y2: conditional branch
        enc[k] = (v[i] << 8) | run;         // Y4: shift and OR, Y5: store
        k++;
        i += run;                           // Y4: addition
    }
    return k;
}

// Y3: takes a parameter, returns a value. Expands enc[0..len-1] into dec[], returns element count
s32 rle_decode(s32 len)
{
    s32 o = 0;
    for (s32 j = 0; j < len; j++) {
        s32 value = enc[j] >> 8;            // Y4: shift
        s32 run = enc[j] & 0xFF;            // Y4: AND
        for (s32 t = 0; t < run; t++) {
            dec[o++] = value;               // Y5: store
        }
    }
    return o;
}

int main(void)
{
    // LCG input generation: x0 = SID, x = 1664525*x + 1013904223 (mod 2^32)
    u32 x = (u32)SID;                       // Y6: SID seeds the generator
    for (s32 i = 0; i < N; i++) {           // Y1: 1st pass over v
        x = 1664525u * x + 1013904223u;     // Unsigned 32-bit overflow is defined behaviour in C so it gives mod 2^32 automatically
        v[i] = (s32)((x >> 16) % R) + B;    // Y5: store to array
    }
    enc_len = rle_encode(N);              // Y3: parameterised call, value returned
    s32 out = rle_decode(enc_len);      // Y3: parameterised call, value returned
    ratio_x100 = (100 * enc_len) / N;       // Y4: multiplication
    decode_ok = (out == N);
    for (s32 i = 0; i < N; i++) {           // Y1: 3rd pass over v
        if (dec[i] != v[i]) {               // Y2: conditional branch
            decode_ok = 0;
```

```
67          }
68      }
69      return 0;
70  }
```

Tek dosya, C99, 54 satır kod (toplam 70 satır; 9 boş ve 6 yalnızca yorum içeren satır hariç). `clang -std=c99 -pedantic-errors -Wall -Wextra` ile uyarısız derlenir. Uzun yorum satırları listede ↪ ile sarılmıştır.

1.4 Zorunlu yapı eşlemesi. Her madde için satır numarası ve tek cümlelik açıklama:

#	Zorunlu yapı	Satır no	Açıklama
Y1	N elemanlı global int dizisi, en az iki geçiş	**15**; 56, 28, 64	`s32 v[N];` — `v` üzerinde üç geçiş: s. 56 yazma, s. 28 okuma/kodlama, s. 64 doğrulama
Y2	Koşullu dallanma	**30**; 65	s. 30 `v[i+run] == v[i]` koşu uzunluğunu belirler; s. 65 `if (dec[i] != v[i])`
Y3	Parametreli, değer döndüren fonksiyon çağrısı	**24**, 39 (tanım); **60**, 61 (çağrı)	`s32 rle_encode(s32 n)` ve `s32 rle_decode(s32 len)`; ikisi de parametre alır, değer döndürür ve dönen değer çağrı yerinde bir değişkene atanır
Y4	Toplama/çıkarma, çarpma veya shift, bit işlemi	**31**, 33, 43, 44, 62	s. 31 `<< 8` ve `|`; s. 33 `i += run`; s. 43 `>> 8`; s. 44 `& 0xFF`; s. 62 `100 * enc_len`
Y5	Dizi elemanına store	**58**, 31, 46	`v[i] = ...` (s. 58), `enc[k] = ...` (s. 31), `dec[o++] = ...` (s. 46)
Y6	`const int SID` ve hesapta kullanımı	**13** (tanım), **55** (kullanım)	`const int SID = 1230025;` (numara $2^{31}-1$'i aştığı için son 9 hane); LCG'nin x_0 tohumudur, dolayısıyla tüm `v` dizisini ve üç Y7 çıktısını belirler
Y7	Sonucun tutulduğu global değişken(ler)	**19, 20, 21**	`enc_len`, `ratio_x100`, `decode_ok`

1.5 Program çıktıları. Simülatörde çalıştırıldıktan sonra bellekten okunan değerler:

Global değişken	Adres (hex)	Değer	Ekran görüntüsü dosyası
`enc_len`	`0x140075C60`	**11** (`0x0000000B`)	`ss_globals.png`
`ratio_x100`	`0x140075C64`	**45** (`0x0000002D`)	`ss_globals.png`
`decode_ok`	`0x140075C68`	**1** (`0x00000001`)	`ss_globals.png`
`v[24]`	`0x140075C70`	`0001 0001 0000 0111 0010 1110`	`ss_globals.png`
`enc[0..10]`	`0x140075CD0`	`0x0003 0x0101 0x0003 0x0101 0x0005 0x0103 0x0002 0x0101 0x0001 0x0103 0x0001`	`ss_globals.png`
`dec[24]`	`0x140075D30`	`v` ile birebir aynı	`ss_globals.png`

Değerler Visual Studio 2022 Watch penceresinden okunmuştur; pencere onlu taban kullanır (`257` = `0x0101`, `259` = `0x0103`) ve `enc`'in tanımlı 24 elemanının tamamını gösterir, yalnızca ilk `enc_len` = 11 tanesi anlamlıdır. `ratio_x100` = $\lfloor 100 \times 11/24 \rfloor = 45$; yönergedeki $R = 2$ için "yaklaşık 50" beklentisine göre fark, $N = 24$'ün küçüklüğünden kaynaklanan örneklem dalgalanmasıdır (beklenen koşu sayısı 12, ölçülen 11).

1.6 İzlenecek kernel (12–20 dinamik instruction; en az bir load, store, ALU, koşullu branch ve call/return içermeli):

Alan	Değer
C satır aralığı	s. 49 (`return o;`) ve s. 61–64
Başlangıç ve bitiş adresi (hex)	`0x140001168` – `0x140001232` (toplam 68 byte)
Dinamik instruction sayısı	**17**
Hangi döngü iterasyonu (ör. i = 3)	`rle_decode`'dan tek dönüş; ardından doğrulama döngüsünün ilk testi ($i = 0$)

İçerik kontrolü: load (#1, 5, 10, 16), store (#4, 9, 14), ALU (#5 `imul`, #8 `idiv`, #12 `and`), koşullu branch (#17 `jge`), fonksiyon dönüşü (#3 `ret`). $k = 7$ interrupt'ı 7. instruction olan `cdq`'ya (`0x140001208`) denk gelir; kaydedilecek PC = `0x140001209`. Kernel'in tam instruction listesi ek belgededir (`M1_Adim0_Adim1.pdf`, Bölüm 2.6).

Öğretim üyesi (M1): ☐ Onaylandı ☐ Düzeltme istendi ☐ Reddedildi — Tarih: _____ Not: _____

Adım 2 — Assembly ve machine code (M2: 9 Kasım, Pazartesi)

2a. C → assembly eşlemesi (-O0; listenin tamamı depoda asm_O0.s):

Derleme: `cmake --preset od` (`/Zi /Od /Ob0`). Her derleme `listings/` altına derleyicinin C satır işaretli assembly'sini (`asm_O0.s`) ve linklenmiş exe'deki kendi fonksiyonlarımızın disassembly'sini (`objdump_O0.txt`) yazar. İkisi instruction instruction eşlenerek her C satırının gerçek adres aralığı bulunmuştur; 146 instruction'ın tamamı `listings/c2asm_O0.txt` dosyasındadır.

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)
55	`u32 x = (u32)SID;`	`mov dword ptr [rsp+0x30], 0x12c4c9`	`0x14000118C`–`0x140001193`
57	`x = 1664525u * x + 1013904223u;`	`imul eax, dword ptr [rsp+0x30], 0x19660d; add eax, 0x3c6ef35f; mov dword ptr [rsp+0x30], eax`	`0x1400011A3`–`0x1400011B3`
28	`while (i < n) {`	test: `mov eax, [rsp+0x10]; cmp eax, [rsp+0x14]; jge 0x1400010C5` geri dönüş: `jmp 0x140001018`	`0x140001018`–`0x140001025` ve `0x1400010C0`–`0x1400010C4`
31	`enc[k] = (v[i] << 8) | run;`	`movsxd rcx, [rsp+0x10]; lea rax, [rip+..]` (v); `mov edx, [rax+4*rcx]; shl edx, 0x8; or edx, [rsp+0x8]; movsxd rcx, [rsp+0xc]; lea rax, [rip+..]` (enc); `mov [rax+4*rcx], edx`	`0x140001084`–`0x1400010A8`
60	`enc_len = rle_encode(N);`	`mov ecx, 0x18; call 0x140001000; mov [rip+..], eax` (enc_len)	`0x1400011DD`–`0x1400011EC`
62	`ratio_x100 = (100 * enc_len) / N;`	`imul eax, [rip+..], 0x64; mov ecx, 0x18; cdq; idiv ecx; mov [rip+..], eax`	`0x1400011FC`–`0x140001210`

Satır 28 iki ayrı adres aralığına düşer: döngü testi döngünün başında, geri dönüş `jmp`'i sonundadır; tek bir C satırı bitişik olmayan makine koduna karşılık gelebilir. Satır 30'daki `&&` koşulu -O0'da 23 instruction'a açılır: önce sınır testi (`jge`), ardından `cmp` + `sete` ile sonuç bir byte'lık stack değişkenine yazılıp tekrar okunur.

Bölüm	Başlangıç adresi (hex)	Boyut (byte)	İçerik
`.text`	`0x140001000`	395 750 (`0x609E6`)	Kendi kodumuz `0x140001000`–`0x140001272` (627 byte: `rle_encode`, `rle_decode`, `main`); geri kalanı statik CRT
`.rdata`	`0x140062000`	75 452 (`0x126BC`)	`SID` (`0x140062000`) ve CRT'nin salt-okunur verisi
`.data` / `.bss`	`0x140075000`	3 584 (`0xE00`)	`enc_len 0x140075C60`, `ratio_x100`, `decode_ok`, `v 0x140075C70`, `enc 0x140075CD0`, `dec 0x140075D30`. Sıfırla ilklenen globaller; PE'de ayrı `.bss` bölümü yoktur, linker bunları `.data`'ya katar.

Von Neumann ilkesi bu adreslerde nasıl görünüyor? Komutlar (`.text`, `0x140001000`) ve veriler (`.rdata 0x140062000`, `.data 0x140075000`) aynı tek ve doğrusal adres uzayındadır. Bu RIP-relative adreslemede doğrudan görülür: `0x1400011C6`'daki `lea rax, [rip+0x74AA3]`, `v`'nin adresini bir sonraki komutun adresine sabit bir uzaklık ekleyerek bulur (`0x1400011CD` + `0x74AA3` = `0x140075C70`); bu hesap ancak komut ve veri aynı adres uzayında durduğunda anlamlıdır.

Program `SID`'i `.rdata`'dan okumaz: `const int` olduğu için derleyici değeri satır 55'teki komutun içine immediate olarak gömer (`c9 c4 12 00`). `.rdata`'daki kopya yalnızca `SID`'in dış bağlantılı (external linkage) bir sembol olması nedeniyle üretilir.

2b. -O0 ve -O1 karşılaştırması (kernel için):

Kernel C satırlarıyla tanımlıdır (s. 49 `return o;` ve s. 61–64); `/O1` sütunu aynı C satırlarına ait instruction'ları sayar (`listings/c2asm_O1.txt`). Load ve store sayıları bütün bellek okuma/yazmalarını içerir; `ret` ve `pop` stack'ten okudukları için load sayılır.

Ölçü	-O0	-O1	Farkın nedeni
Kernel'deki instruction sayısı	17	18	Register allocation spill/reload'ları kaldırdı (`out` ve `i` artık register'da; s. 61: 1→0) ve döngü döndürüldüğü için ilk test silindi (`N` = 24 > 0 derleme anında bilinir; s. 64: 3→0). Buna karşılık `/ 24` bölmesi `cdq`+`idiv` yerine sabitle çarpma dizisine dönüştü (`imul` ile `0x2AAAAAAB`, `sar 34`; s. 62: 5→8) ve `rle_decode` dönüşte üç callee-saved register'ı geri yükledi (s. 49: 3→4). Net etki: daha az bellek erişimi, biraz daha fazla ama daha ucuz instruction.
Load sayısı	5	5	-O0: 3 spill okuma (`o`, `out`, `i`) + `enc_len` + `ret`. -O1: spill okuma 0; yerine 3 `pop` (callee-saved `rbx`, `rdi`, `rsi`) + `enc_len` + `ret`. Sayı aynı, içerik farklı: değişken trafiği tamamen kalktı.
Store sayısı	4	2	-O0'da `out` ve `i` stack'e yazılır (2 spill store); -O1'de register'da kalırlar. Kalan 2 store `ratio_x100` ve `decode_ok`'tur ve kaldırılamaz: sonuç global değişkende tutulmalıdır (Y7).
Kullanılan register sayısı	4	7	-O0 her değişkeni stack'te tuttuğu için yalnızca `rax`, `rcx`, `rdx`, `rsp` kullanır. -O1 değişkenleri register'larda yaşatır: `rax`, `rcx`, `rdx`, `rsp` + `rbx`, `rdi`, `rsi`. Bu üçü callee-saved olduğu için kullanılmaları onları kaydetme/geri yükleme maliyetini (`push`/`pop`) beraberinde getirir.

Bellek referansı toplamı (load + store): -O0'da 9, -O1'de 7. Stack üzerindeki değişken trafiği 5'ten 0'a iner; azalmanın tamamı register allocation'dan gelir.

2c. Elle machine code (en az 10 instruction; x86-64 için 8). Kullanılan tüm formatlar kapsanmalı.

Değişken uzunluklu x86-64'te "format", bir instruction'ın hangi alanları taşıdığıdır (prefix, REX, opcode, ModR/M, SIB, displacement, immediate). Format sütunu Intel SDM Cilt 2'nin "Op/En" sınıflandırmasını

kullanır. Seçilen 10 instruction programda kullanılan her alan türünü en az bir kez içerir: REX.W (#2), tek byte'lık opcode (çoğu), iki byte'lık `0F` opcode (#9), ModR/M.reg içinde opcode uzantısı (#2, #4), opcode içinde register (#8), ModR/M mod = 00 RIP-relative (#7), mod = 01 (#1, #4, #6), mod = 11 (#2, #9), SIB (#1, #4, #6), disp8 (#1, #4, #6), disp32 (#7), imm8 (#2, #7), imm32 (#4, #8), rel8 (#10), rel32 (#5) ve operandsız (#3). Instruction'ların 8'i kernel'den alınmıştır (#4 ve #5 hariç); doğrulama `listings/objdump_O0.txt` ile yapılır.

#	Adres (hex)	Assembly	Format	Alanlar (ad = binary)	Machine code (hex)	Bellekteki byte sırası	objdump ile aynı mı?

1	`0x140001168`	`mov eax, dword ptr [rsp+0x10]`	RM: `8B /r`	`op=10001011`
				`mod=01 reg=000`
				`rm=100`
				`ss=00 idx=100`
				`base=100`
				`disp8=00010000`
					`8B 44 24 10`	`8B, 44, 24, 10`	✓

2	`0x14000116C`	`add rsp, 0x18`	MI: `REX.W 83 /0 ib`	`REX=0100 W=1 R=0 X=0 B=0`
				`op=10000011`
				`mod=11 reg=000(/0)`
				`rm=100`
				`imm8=00011000`
					`48 83 C4 18`	`48, 83, C4, 18`	✓

3	`0x140001170`	`ret`	ZO: `C3`	`op=11000011`	`C3`	`C3`	✓

4	`0x14000118C`	`mov dword ptr [rsp+0x30], 0x12c4c9`	MI: `C7 /0 id`	`op=11000111`
				`mod=01 reg=000(/0)`
				`rm=100`
				`ss=00 idx=100`
				`base=100`
				`disp8=00110000`
				`imm32=0x0012C4C9`
					`C7 44 24 30 C9 C4 12 00`	`C7, 44, 24, 30, C9, C4, 12, 00 (imm32 LE)`	✓

5	`0x1400011F3`	`call 0x1400010D0 (rle_decode)`	D: `E8 cd`	`op=11101000` `rel32=0xFFFFFED8 (-296)`	`E8 D8 FE FF FF`	`E8, D8, FE, FF, FF (rel32 LE)`	✓

6	`0x1400011F8`	`mov dword ptr [rsp+0x28], eax`	MR: `89 /r`	`op=10001001`
				`mod=01 reg=000`
				`rm=100`
				`ss=00 idx=100`
				`base=100`
				`disp8=00101000`
					`89 44 24 28`	`89, 44, 24, 28`	✓

7	`0x1400011FC`	`imul eax, dword ptr [rip+0x74a5d], 0x64`	RMI: `6B /r ib`	`op=01101011`
				`mod=00 reg=000`
				`rm=101(RIP)`
				`disp32=0x00074A5D`
				`imm8=01100100`
					`6B 05 5D 4A 07 00 64`	`6B, 05, 5D, 4A, 07, 00, 64 (disp32 LE)`	✓

8	`0x140001203`	`mov ecx, 0x18`	OI: `B8+rd id`	`op=10111`
				`reg=001(ecx)`
				`imm32=0x00000018`
					`B9 18 00 00 00`	`B9, 18, 00, 00, 00 (imm32 LE)`	✓

9	`0x140001216`	`sete al`	M: `0F 94`	`esc=00001111`
				`op=10010100`
				`mod=11 reg=000`
				`rm=000(al)`
					`0F 94 C0`	`0F, 94, C0`	✓

10	`0x140001231`	`jge 0x14000126C`	D: `7D cb`	`op=01111101` `rel8=00111001 (+57)`	`7D 39`	`7D, 39`	✓

Doğrulama. Tablodaki 10 instruction'ın makine kodu, linklenmiş `build-od/rle.exe` üzerinde Visual Studio 2022 Disassembly penceresinde (*Show Code Bytes* açık) birebir doğrulanmıştır; ekran görüntüsü depoda `ss_m2_disasm.png` dosyasındadır. Pencere C satırlarını da gösterdiği için 2a'daki eşlemeyi de doğrular; stack değişkenleri PDB'den gelen adlarıyla görünür (ör. `[x]` = `[rsp+0x30]`, `[out]` = `[rsp+0x28]`).

Zorunlu instruction'ların tablodaki sıra numarası:

- ☒ SID'i yükleyen instruction(lar): # 4
- ☒ Koşullu branch: # 10

- ☒ Load: # 1
- ☒ Store: # 6
- ☒ Fonksiyon çağrısı: # 5

SID'in yüklenmesi. Sabitin neden tek instruction'a sığmadığı, nasıl bölündüğü ve gerekiyorsa +1 düzeltmenin sign extension ile açıklaması:

x86-64'te sabit tek instruction'a **sığar**, bölünmez ve düzeltme gerekmez. #4 (`C7 /0 id`) 32 bitlik bir immediate alanı taşır; `SID` $= 1\,230\,025 =$ `0x0012C4C9`, instruction'ın son dört byte'ına little-endian olarak yazılır: `C9 C4 12 00`. Bu, değişken uzunluklu encoding sayesinde mümkündür: instruction sabiti taşımak için 8 byte'a uzar. (`/O1`'de aynı sabit doğrudan register'a yüklenir: `mov eax, 0x12C4C9` = `B8 C9 C4 12 00`, yine tek instruction.)

Sabit 32 bit uzunluklu RV32I'de ise 32 bitlik bir sabit, 32 bitlik instruction'ın içine opcode ve register alanlarıyla birlikte sığmaz. Sabit ikiye bölünür: `lui` üst 20 biti, `addi` alt 12 biti yükler. `addi`'nin 12 bitlik immediate'i sign-extend edildiği için, alt 12 bitin en üst biti (bit 11) 1 olsaydı bu değer negatif yorumlanır ve sonuçtan 2^{12} düşerdi; bunu dengelemek için `lui` değerine $+1$ eklenirdi. `SID` için alt 12 bit `0x4C9` = `0100 1100 1001`, bit $11 = 0$ olduğundan düzeltme gerekmez: `lui 0x12C` ($\rightarrow$ `0x0012C000`) + `addi 0x4C9` = `0x0012C4C9`.

Branch offset hesabı. Branch'in adresi, hedef adres ve offset'in adım adım hesabı:

#10 `jge`, PC-relative'dir ve offset bir **sonraki** instruction'ın adresinden ölçülür (CPU branch'i execute ederken PC zaten bir sonraki instruction'ı gösterir).

1. Branch'in adresi: `0x140001231`; uzunluğu 2 byte (`7D` + rel8).
2. Bir sonraki instruction'ın adresi: `0x140001231` + 2 = `0x140001233`.
3. Hedef adres: `0x14000126C` (`xor eax, eax`, döngüden çıkış; s. 69).
4. Offset = hedef − sonraki = `0x26C` − `0x233` = `0x39` = $+57$.
5. $-128 \leq 57 \leq 127$ olduğu için 8 bitlik alan yeterli: rel8 = `0011 1001` = `0x39`. Machine code: `7D 39`.
6. Çalışırken: koşul (SF = OF, yani `i` ≥ 24) sağlanırsa PC $\leftarrow$ `0x140001233` + `0x39` = `0x14000126C`; sağlanmazsa PC `0x140001233`'te kalır.

Geriye doğru örnek (#5 `call`): sonraki adres `0x1400011F3` + 5 = `0x1400011F8`, hedef `0x1400010D0`; offset = `0x10D0` − `0x11F8` = $-$`0x128` = -296. 32 bitlik iki'ye tümleyen: `0xFFFFFED8`, little-endian `D8 FE FF FF`.

Adım 3 — CPU, interrupt ve bus (final paketiyle: 16 Kasım, Pazartesi)

Model varsayımları (okudum ve uyguladım):

- ☒ Pipeline yok, her an tek instruction yürütülür.
- ☒ PC, IR, MAR ve MBR Fig. 3.2'deki gibi kavramsal olarak vardır.
- ☒ Her bellek erişimi bir bus işlemidir; cache bu adımda yok sayılır.
- ☒ Data bus genişliği = **64** bit. Instruction fetch, Fig. 3.2 modeline uygun olarak instruction başına bir bus işlemi sayılır (kernel'deki en uzun instruction 8 byte'tır, 64 bit'e sığar).

Modelin gerçek donanımdan ayrıldığı yerler (3–5 cümle): x86-64'te MAR, MBR ve IR mimari register değildir; debugger'da yalnızca PC (`RIP`) ve genel register'lar görülür, tablodaki MAR/MBR değerleri instruction semantiğinden türetilmiştir. Gerçek çekirdek (i7-12700H) out-of-order ve superscalar çalışır: bu 17 instruction birkaç cycle'da biter, `cmp`+`jge` makro-füzyonla tek işlem olur, bellek operandlı instruction'lar mikro-op'lara bölünür. Instruction fetch tek tek değil 16–32 byte'lık bloklar halinde yapılır ve değişken uzunluklu instruction'lar bu bloklardan çözülür; modeldeki "instruction başına bir fetch" bu

yüzden bir basitleştirmedir. Gerçekte her erişim önce 48 KB L1D'ye gider (64 byte line), bus'a yalnızca miss'ler çıkar; donanım prefetcher'ı da sıralı erişimleri önceden getirir.

3a. Instruction cycle izleme tablosu. Her dinamik instruction bir satırdır. Bir instruction birden çok bellek erişimi yapıyorsa MAR, MBR ve bus hücrelerine sırayla yazılır. Kategori: PM = processor–memory, PIO = processor–I/O, DP = data processing, C = control. Bus sütununda MR = Memory read, MW = Memory write. Register değerleri `build-od/rle.exe` debugger'da adım adım yürütülerek okunmuştur (`listings/exec_O0.csv`).

#	PC (önce → sonra)	Assembly	IR (hex)	Fig. 3.6 durumları	MAR	MBR	ALU işlemi	Değişen register	Bus (adres / data / control)	Kat.
1	168 → 16C	mov eax, [rsp+0x10]	8B442410	iac, if, iod, oac, of	0x140001168; 0x15FED0	8B442410; 0x00000018	yok (adres = RSP+0x10)	EAX=0x18	(0x140001168 / 8B442410 / MR), (0x15FED0 / 0x18 / MR)	PM
2	16C → 170	add rsp, 0x18	4883C418	iac, if, iod, do	0x14000116C	4883C418	0x15FEC0+0x18	RSP=0x15FED8	(0x14000116C / 4883C418 / MR)	DP
3	170 → 1F8	ret	C3	iac, if, iod, oac, of	0x140001170; 0x15FED8	C3; 0x1400011F8	RSP+8	RIP=0x1400011F8, RSP=0x15FEE0	(0x140001170 / C3 / MR), (0x15FED8 / 0x1400011F8 / MR)	C (PM)
4	1F8 → 1FC	mov [rsp+0x28], eax	89442428	iac, if, iod, oac, os	0x1400011F8; 0x15FF08	89442428; 0x00000018	yok (adres = RSP+0x28)	bellek: out=0x18	(0x1400011F8 / 89442428 / MR), (0x15FF08 / 0x18 / MW)	PM
5	1FC → 203	imul eax, [rip+0x74A5D], 0x64	6B055D4A070064	iac, if, iod, oac, of, do	0x1400011FC; 0x140075C60	6B055D4A070064; 0x0000000B	$11 \times 100 = 1100$	EAX=0x44C	(0x1400011FC / 6B055D4A070064 / MR), (0x140075C60 / 0x0B / MR)	DP (PM)
6	203 → 208	mov ecx, 0x18	B918000000	iac, if, iod, do	0x140001203	B918000000	yok (immediate → ECX)	ECX=0x18	(0x140001203 / B918000000 / MR)	DP
7	208 → 209	cdq **($k = 7$)**	99	iac, if, iod, do	0x140001208	99	EAX işaret genişletme → EDX	EDX=0x0	(0x140001208 / 99 / MR)	DP
8	209 → 20B	idiv ecx	F7F9	iac, if, iod, do	0x140001209	F7F9	$1100 \div 24 = 45$, kalan 20	EAX=0x2D, EDX=0x14	(0x140001209 / F7F9 / MR)	DP
9	20B → 211	mov [rip+0x74A53], eax	8905534A0700	iac, if, iod, oac, os	0x14000120B; 0x140075C64	8905534A0700; 0x0000002D	yok	bellek: ratio_x100=45	(0x14000120B / 8905534A0700 / MR), (0x140075C64 / 0x2D / MW)	PM
10	211 → 216	cmp [rsp+0x28], 0x18	837C242818	iac, if, iod, oac, of, do	0x140001211; 0x15FF08	837C242818; 0x00000018	$24 - 24 = 0$	RFLAGS=0x246 (ZF=1)	(0x140001211 / 837C242818 / MR), (0x15FF08 / 0x18 / MR)	DP (PM)
11	216 → 219	sete al	0F94C0	iac, if, iod, do	0x140001216	0F94C0	AL ← ZF	AL=1	(0x140001216 / 0F94C0 / MR)	DP
12	219 → 21B	and al, 0x1	2401	iac, if, iod, do	0x140001219	2401	$1 \wedge 1 = 1$	RFLAGS=0x202	(0x140001219 / 2401 / MR)	DP
13	21B → 21E	movzx eax, al	0FB6C0	iac, if, iod, do	0x14000121B	0FB6C0	sıfırla genişletme	EAX=0x1	(0x14000121B / 0FB6C0 / MR)	DP
14	21E → 224	mov [rip+0x74A44], eax	8905444A0700	iac, if, iod, oac, os	0x14000121E; 0x140075C68	8905444A0700; 0x00000001	yok	bellek: decode_ok=1	(0x14000121E / 8905444A0700 / MR), (0x140075C68 / 0x1 / MW)	PM
15	224 → 22C	mov [rsp+0x24], 0x0	C7442424 00000000	iac, if, iod, oac, os	0x140001224; 0x15FF04	C7442424 00000000; 0x00000000	yok	bellek: i=0	(0x140001224 / C7442424 00000000 / MR), (0x15FF04 / 0x0 / MW)	PM
16	22C → 231	cmp [rsp+0x24], 0x18	837C242418	iac, if, iod, oac, of, do	0x14000122C; 0x15FF04	837C242418; 0x00000000	$0 - 24$: SF=1, OF=0	RFLAGS=0x297	(0x14000122C / 837C242418 / MR), (0x15FF04 / 0x0 / MR)	DP (PM)
17	231 → 233	jge 0x14000126C	7D39	iac, if, iod	0x140001231	7D39	koşul SF=OF? hayır → alınmaz	RIP=0x140001233 (sıralı)	(0x140001231 / 7D39 / MR)	C

PC sütununda adreslerin son üç hanesi yazılmıştır (tamamı `0x140001xxx`).

Kernel toplamları	Sayı
Instruction fetch	**17**
Operand fetch (of)	**5** (#1, 3, 5, 10, 16)
Operand store (os)	**4** (#4, 9, 14, 15)
PC'nin sıra dışı değiştiği instruction	**1** (#3 `ret`)

PC'nin sıra dışı değişmesi. 15 instruction'da PC ← PC + instruction uzunluğu olur. #3 `ret`'te PC, stack'ten okunan dönüş adresiyle değişir: PC ← [`0x15FED8`] = `0x1400011F8`; bu değeri `0x1400011F3`'teki `call` oraya yazmıştı. #17 `jge` koşullu bir sıçramadır; koşul sağlansaydı PC ← `0x140001233` + `0x39` = `0x14000126C` olurdu, ancak $i = 0 < 24$ olduğundan (SF ≠ OF) alınmaz ve PC sıralı olarak `0x140001233`'e geçer.

3b. Fig. 3.4 makinesine aktarım. Kernel'in kendisi bir dizi elemanını güncellemez (store'ları `out`, `i`, `ratio_x100` ve `decode_ok` skalerlerine gider). Bu yüzden kernel'in bulunduğu `rle_decode` fonksiyonundaki dizi güncellemesi alınmıştır: s. 46 `dec[o++] = value;`, $o = 3$ iterasyonu (`value` = 1). Fig. 3.4 makinesinde 16 bitlik word, 4 bit opcode + 12 bit adres vardır; bellek yerleşimi: program `300`'den, `value` `900`, `o` `901`, sabit 1 `902`, `dec` `910`'dan başlar (`dec[3]` = `913`). Kullanılan opcode'lar: 1 = AC ← bellek, 2 = bellek ← AC, 5 = AC ← AC + bellek. Program I/O yapmadığı için Problem 3.1'in 3 ve 7 (I/O) opcode'larına gerek yoktur. Program: `1900` (LOAD value), `2913` (STORE dec[3]), `1901` (LOAD o), `5902` (ADD 1), `2901` (STORE o).

Adım	Fetch / execute	PC	IR (hex)	AC	Değişen bellek hücresi
1	Fetch	`300`→`301`	`1900`	—	—
2	Execute	`301`	`1900`	`0001`	— (AC ← M[`900`])
3	Fetch	`301`→`302`	`2913`	`0001`	—
4	Execute	`302`	`2913`	`0001`	M[`913`] = `0001` (`dec[3]`)
5	Fetch	`302`→`303`	`1901`	`0001`	—
6	Execute	`303`	`1901`	`0003`	— (AC ← M[`901`])
7	Fetch	`303`→`304`	`5902`	`0003`	—
8	Execute	`304`	`5902`	`0004`	— (AC ← `0003` + `0001`)
9	Fetch	`304`→`305`	`2901`	`0004`	—
10	Execute	`305`	`2901`	`0004`	M[`901`] = `0004` (`o`)

Ölçü	Seçilen ISA	Fig. 3.4 makinesi
Aynı iş için instruction cycle sayısı	8 (-O0, `0x140001130`–`0x14000114B`)	5

x86-64 burada daha fazla instruction kullanır çünkü -O0, `o`'nun eski değerini indis olarak saklamak için bir register kopyası yapar ve `dec[o]`'nun adresini çalışma anında hesaplar (`movsxd`, `lea`, indisli store). Fig. 3.4 makinesinde indisli adresleme yoktur; yukarıdaki program yalnızca $o = 3$ için çalışır. Gerçek bir döngüde STORE instruction'ının adres alanını kendisi değiştiren (self-modifying) kod gerekirdi.

Tek accumulator'ın getirdiği sınırlama (PDP-11 ADD B,A ile karşılaştırarak): Fig. 3.4 makinesinde her işlem AC üzerinden geçmek zorundadır; bellekteki bir değeri artırmak (`o++`) LOAD, ADD ve STORE olmak üzere üç instruction, üç instruction fetch ve üç operand erişimi gerektirir ve AC'deki önceki değer kaybolur. PDP-11'in `ADD B,A`'sı iki operandlı bir instruction'dır: iki operand da register veya bellekte olabilir, toplam doğrudan hedefe yazılır; aynı iş tek instruction'dır ve ara değer için accumulator gerekmez. x86-64 de iki operandlıdır ve bir operandı bellekte olabilir (`add dword ptr [rsp+0x10], 1` geçerli bir instruction'dır); -O0'ın bunu dört instruction'la yapması derleyici seçimidir, ISA sınırlaması değildir.

IAS register'ları ve seçilen ISA:

IAS register'ı	IAS'taki görevi	Seçilen ISA'daki karşılığı	Programcıya görünür mü?
MBR	Belleğe yazılacak / bellekten okunan word'ü tutar	Mimari karşılığı yok; load/store tamponları (iç yapı)	Hayır
MAR	Okunacak / yazılacak word'ün adresini tutar	Mimari karşılığı yok; etkin adresi AGU hesaplar (iç yapı)	Hayır
IR	Yürütülen instruction'ın 8 bitlik opcode'u	Mimari karşılığı yok; decoder instruction'ı mikro-op'lara çözer	Hayır
IBR	Bir word'deki sağdaki instruction'ı geçici tutar	Front-end'in instruction kuyruğu / fetch tamponu	Hayır
PC	Sonraki instruction çiftinin adresi	`RIP`	Kısmen: `mov` ile yazılamaz; `jmp`/`call`/`ret` değiştirir, `lea [rip+..]` okur
AC	ALU işlemlerinin operandı ve sonucu	`RAX`/`EAX`; tarihsel accumulator, kısa encoding'leri vardır (`0x1400011AB: add eax, imm32` = `05 id`)	Evet
MQ	Çarpımın düşük yarısı, bölümün sonucu	`RDX:RAX` çifti; `0x140001208`–`09`'daki `cdq`/`idiv`: `EDX:EAX` bölünen, `EAX` bölüm, `EDX` kalan	Evet

3c. Interrupt senaryosu ($k = 7$, tür = **Timer**):

Soru	Cevap
Interrupt hangi instruction'ın execute'u sırasında geliyor? (adres, assembly)	`0x140001208 cdq` (kernel'in 7. instruction'ı)
Ne zaman tanınır ve bu instruction neden yarıda kesilmez? (Fig. 3.9)	Interrupt isteği `cdq`'nun execute'u sırasında gelir ama işlemci interrupt hattını yalnızca instruction cycle'ın sonunda, execute bittikten sonra yoklar (Fig. 3.9: interrupt cycle execute'tan sonra gelir). Bu yüzden `cdq` tamamlanır, interrupt instruction sınırında tanınır; yarıda kesilen bir instruction'ın kısmi sonucunu kaydetmek gerekmez.
Kaydedilen PC değeri (hex)	`0x140001209` (sonraki instruction, `idiv ecx`)

Soru	Cevap
Kaydedilen diğer durum bilgisi	İşlemcinin otomatik kaydettikleri: `RFLAGS` (`0x202`), `CS`, `SS`, `RSP` (`0x15FEE0`). Handler kullandığı register'ları kendisi kaydeder; burada kritik olanlar `EDX:EAX` = `0x0:0x44C` (`cdq` az önce hazırladı) ve `ECX` = `0x18`: `idiv` bunlara dayanır, bozulursa `ratio_x100` yanlış çıkar.
Seçilen ISA'daki gerçek mekanizma (register adları, vektör)	Local APIC timer maskelenebilir bir dış interrupt üretir; işlemci vektör numarasıyla IDT'yi indeksler (`IDTR` tabanı + vektör × 16), interrupt gate'ten handler'ın `CS:RIP`'ini yükler. Kullanıcı modundan (CPL 3) çekirdeğe (CPL 0) geçtiği için stack `TSS.RSP0`'a değişir ve 64-bit stack frame'e sırasıyla `SS`, `RSP`, `RFLAGS`, `CS`, `RIP` yazılır. Interrupt gate `RFLAGS.IF`'i temizler. Handler `IRETQ` ile biter; `IRETQ` bu beş değeri geri yükler.
Kaynak (ISA referansı, bölüm, sayfa)	Intel 64 and IA-32 Architectures SDM, Vol. 3A, Order No. 253668-080US (Haziran 2023): §6.10 IDT, s. 6-9; §6.12.1 handler procedures, s. 6-12; §6.12.1.3 IF kullanımı, s. 6-17; §6.14.2 64-Bit Mode Stack Frame, s. 6-20; §6.14.3 IRET in IA-32e Mode, s. 6-21; §11.5.4 APIC Timer, s. 11-16.
Handler dönüşünde ilk çalışan instruction'ın adresi	`0x140001209` (`idiv ecx`)

Fonksiyon çağrısı (Y3) ve interrupt karşılaştırması:

Ölçüt	Fonksiyon çağrısı	Interrupt
Akışı kim başlatır?	Programın kendisi, senkron: `0x1400011F3`'teki `call rle_decode`	Dış donanım olayı, asenkron: APIC timer; programdaki hiçbir instruction'a bağlı değildir
Dönüş adresini kim kaydeder, nereye?	`call` instruction'ı, kullanıcı stack'ine: `0x1400011F8` → `[0x15FED8]` (#3'te `ret` buradan okur)	İşlemci donanımı, çekirdek stack'ine (`TSS.RSP0`): `RIP` = `0x140001209` ile birlikte `RFLAGS`, `CS`, `SS`, `RSP`
User program bundan haberdar mı?	Evet: çağrı kodda açıktır, parametre geçirir ve dönüş değerini (`EAX`) kullanır	Hayır: bağlam aynen geri yüklenir, program kesildiğini fark etmez

İkinci (daha yüksek öncelikli) interrupt. Aynı olay dizisini iki yaklaşımla zaman çizelgesi olarak doldurun (Fig. 3.13 ve 3.14). 1. interrupt Timer, 2. interrupt daha yüksek öncelikli bir I/O cihazı (ör. ağ kartı). x86'da interrupt gate `IF`'i temizlediği için varsayılan davranış sequential'dır; nested için handler `sti` ile interrupt'ları açar ve APIC yalnızca servis edilenden daha yüksek öncelik sınıfındaki interrupt'ı iletir (SDM Vol. 3A §11.8.6, s. 11-31).

Zaman	Sequential (disable)	Nested (priority)
t_0: 1. interrupt gelir	`cdq` biter, bağlam kaydedilir, Timer handler başlar (`IF`=0)	Aynı; handler `sti` ile yüksek öncelikli interrupt'lara izin verir
t_1: 2. interrupt gelir	Bekletilir (APIC'te pending); Timer handler kesintisiz sürer	Timer handler kesilir, onun bağlamı kaydedilir, I/O handler başlar

t_2	Timer handler `IRETQ` ile biter, `IF`=1; bekleyen I/O interrupt hemen tanınır, I/O handler başlar	I/O handler biter, kesilen Timer handler'a dönülür
t_3	I/O handler biter	Timer handler biter
t_4: user program'a dönüş	`0x140001209` `idiv ecx`'ten devam	`0x140001209` `idiv ecx`'ten devam

3d. Bus ve interconnection.

Bus işlemi türü (kernel, cache yok)	Sayı	Aktif control line'lar
Instruction fetch (read)	17	Memory read, Transfer ACK, Clock
Data read	5	Memory read, Transfer ACK, Clock (#1, 3, 5, 10, 16)
Data write	4	Memory write, Transfer ACK, Clock (#4, 9, 14, 15)
Toplam	**26**	

Hesap	Değer	Adımlar
Address bus genişliği ve maksimum adreslenebilir bellek	2^{39} B = 512 GiB	Bu makinenin işlemcisi (i7-12700H, `CPUID 0x80000008`) 39 bit fiziksel ve 48 bit doğrusal adres uygular. Model fiziksel adres bus'ını alır: 2^{39} B. (Doğrusal adres uzayı 2^{48} B = 256 TiB; mimarinin tanımladığı üst sınır 2^{64} B = 16 EiB.)
Data bus 16 bit olsaydı kernel'in bus cycle sayısı	58	Fetch: her instruction $\lceil$uzunluk$/2\rceil$ cycle, 17 instruction (68 byte) $\rightarrow$ 38. Data read: dört 4 byte'lık okuma $\times 2$ + `ret`'in 8 byte'ı $\times 4 = 12$. Data write: $4\times2 = 8$. Toplam $38 + 12 + 8 = 58$.
Mevcut / 16-bit oranı	$26/58 = 0{,}448$	16 bit bus aynı kernel için $58/26 = 2{,}23$ kat bus cycle gerektirir (Problem 3.7 mantığı: genişlik yarıya inince geniş aktarımlar ikiye bölünür).

PCIe paket verimi (N $\times$ 4 = **96** byte'lık tek Memory Write TLP; Fig. 3.25):

Alan (octet)	12-byte header, ECRC yok	16-byte header, ECRC var
STP framing (baş)	1	1
Sequence number	2	2
Header	12	16
Data	96	96
ECRC	0	4
LCRC	4	4
STP framing (son)	1	1
Toplam	**116**	**124**

Verim = data / toplam	$96/116 =$ **%82,76**	$96/124 =$ **%77,42**
128b/130b ile verim	$0{,}8276 \times 128/130 =$ **%81,48**	$0{,}7742 \times 128/130 =$ **%76,23**

Sonucun bir çekirdekten PCIe cihazına yolu (Fig. 3.17 ve 3.21): (1) Çekirdek store'u yürütür; veri çekirdekten QPI bağlantısıyla I/O Hub'a gider. QPI'da protokol katmanı paketi oluşturur, routing katmanı yönlendirir, link katmanı 80 bitlik **flit**'lere böler (72 bit veri + 8 bit CRC), fiziksel katman bunları 20 bitlik **phit**'ler halinde 20 şeritten yollar. (2) I/O Hub, PCIe root complex'tir: isteği bir PCIe Memory Write **TLP**'sine çevirir (transaction katmanı: header + 96 byte veri + isteğe bağlı ECRC). (3) Data link katmanı TLP'ye sequence number ve LCRC ekler; karşı tarafa ACK/NAK **DLLP**'leriyle güvenilir iletim sağlar. (4) Fiziksel katman STP/END framing'i ekler ve 128b/130b kodlu sembolleri şeritlere dağıtır; gerekirse bir PCIe switch üzerinden (5) hedef endpoint'e ulaşır, endpoint katmanları tersine açar. Not: bu makinedeki gibi güncel Intel istemci işlemcilerinde root complex işlemci kalıbının içindedir ve QPI yerine çip içi ring bus kullanılır; Fig. 3.17 sunucu sınıfı çok soketli yapıyı gösterir.

Adım 4 — Cache simülasyonu (final paketiyle: 16 Kasım, Pazartesi)

4a. Data adres izi. Tüm load ve store'lar depoda `trace.csv` dosyasındadır. Biçim: `sira,RW,adres_hex,boyut,c_satiri,stack_mi`. İz, `build-od/rle.exe` debugger'da `main`'den `main`'in `ret`'ine kadar tek adım yürütülerek çıkarılmıştır (`python tools/mktrace.py O0`); etkin adres her instruction'ın bellek operandı ve o anki register değerlerinden hesaplanır. İki ayrı çalıştırmada iz birebir aynıdır. Adres alanları 64 bit adresin düşük 32 biti üzerinden ayrıştırılır.

Ölçü	Değer
Kullanılan derleme (-O0 / -O1)	-O0 (`/Od`)
Toplam data referansı	1152
Load / store	820 / 332
Stack erişimi sayısı	961 (%83,4)
Ana diziye erişim sayısı	`v`: 105 (tüm diziler: `v` 105 + `enc` 33 + `dec` 48 = 186; globaller 5)

4b. İlk 5 farklı adresin alanları: (tag = adres[31:6], set = adres[5:4], word = adres[3:0])

Adres (hex)	Adres (binary: tag | set | word)	Tag	Line / Set	Word
`0x15FF14`	`00000000000101011111111100 01 0100`	`0x57FC`	1 (01)	4 (0100)
`0x15FF10`	`00000000000101011111111100 01 0000`	`0x57FC`	1 (01)	0 (0000)
`0x15FF0C`	`00000000000101011111111100 00 1100`	`0x57FC`	0 (00)	12 (1100)
`0x140075C70`	`01000000000001110101110001 11 0000`	`0x1001D71`	3 (11)	0 (0000)
`0x140075C74`	`01000000000001110101110001 11 0100`	`0x1001D71`	3 (11)	4 (0100)

4c. Elle simülasyon (ilk 30 referans): 256 byte, 16 byte line, 4-way set-associative, LRU, write-through + write-allocate.

#	R/W	Adres (hex)	Tag	Line / Set	Word	Hit / Miss	Yerinden çıkan blok	Bu blok daha önce cache'te miydi?
1	W	`0x15FF14`	`0x57FC`	1	4	Miss	—	hayır (ilk erişim)
2	W	`0x15FF10`	`0x57FC`	1	0	Hit	—	evet

#	R/W	Adres (hex)	Tag	Line / Set	Word	Hit / Miss	Yerinden çıkan blok	Bu blok daha önce cache'te miydi?
3	W	`0x15FF0C`	`0x57FC`	0	12	Miss	—	hayır (ilk erişim)
4	R	`0x15FF0C`	`0x57FC`	0	12	Hit	—	evet
5	R	`0x15FF10`	`0x57FC`	1	0	Hit	—	evet
6	W	`0x15FF10`	`0x57FC`	1	0	Hit	—	evet
7	R	`0x15FF10`	`0x57FC`	1	0	Hit	—	evet
8	R	`0x15FF0C`	`0x57FC`	0	12	Hit	—	evet
9	W	`0x140075C70`	`0x1001D71`	3	0	Miss	—	hayır (ilk erişim)
10	R	`0x15FF0C`	`0x57FC`	0	12	Hit	—	evet
11	W	`0x15FF0C`	`0x57FC`	0	12	Hit	—	evet
12	R	`0x15FF0C`	`0x57FC`	0	12	Hit	—	evet
13	R	`0x15FF10`	`0x57FC`	1	0	Hit	—	evet
14	W	`0x15FF10`	`0x57FC`	1	0	Hit	—	evet
15	R	`0x15FF10`	`0x57FC`	1	0	Hit	—	evet
16	R	`0x15FF0C`	`0x57FC`	0	12	Hit	—	evet
17	W	`0x140075C74`	`0x1001D71`	3	4	Hit	—	evet
18	R	`0x15FF0C`	`0x57FC`	0	12	Hit	—	evet
19	W	`0x15FF0C`	`0x57FC`	0	12	Hit	—	evet
20	R	`0x15FF0C`	`0x57FC`	0	12	Hit	—	evet
21	R	`0x15FF10`	`0x57FC`	1	0	Hit	—	evet
22	W	`0x15FF10`	`0x57FC`	1	0	Hit	—	evet
23	R	`0x15FF10`	`0x57FC`	1	0	Hit	—	evet
24	R	`0x15FF0C`	`0x57FC`	0	12	Hit	—	evet
25	W	`0x140075C78`	`0x1001D71`	3	8	Hit	—	evet
26	R	`0x15FF0C`	`0x57FC`	0	12	Hit	—	evet
27	W	`0x15FF0C`	`0x57FC`	0	12	Hit	—	evet
28	R	`0x15FF0C`	`0x57FC`	0	12	Hit	—	evet
29	R	`0x15FF10`	`0x57FC`	1	0	Hit	—	evet
30	W	`0x15FF10`	`0x57FC`	1	0	Hit	—	evet

İlk 30 referans `main`'in başı ve LCG döngüsünün ilk dört iterasyonudur: iki stack slot'u (`x` ve `i`) sürekli yeniden okunur ve yazılır. Bu yüzden yalnızca üç compulsory miss vardır (stack bloğu `0x15FF10`, stack bloğu `0x15FF00` ve `v`'nin ilk bloğu `0x40075C70`); `v[1]` ve `v[2]` aynı bloğa düştüğü için hit olur (spatial locality).

Cache'in 30. referanstan sonraki anı. 16 line'ın tamamı yazılmıştır (line = set $\times$ 4 + yol). Write-through olduğu için dirty biti yoktur.

Line	Set	Valid	Tag (hex)	Dirty (write-back)	LRU sırası (associative; 0 = en yeni)
0	0	1	`0x57FC`	— (write-through)	0
1	0	0	—	— (write-through)	—
2	0	0	—	— (write-through)	—

Line	Set	Valid	Tag (hex)	Dirty (write-back)	LRU sırası (associative; 0 = en yeni)
3	0	0	—	— (write-through)	—
4	1	1	`0x57FC`	— (write-through)	0
5	1	0	—	— (write-through)	—
6	1	0	—	— (write-through)	—
7	1	0	—	— (write-through)	—
8	2	0	—	— (write-through)	—
9	2	0	—	— (write-through)	—
10	2	0	—	— (write-through)	—
11	2	0	—	— (write-through)	—
12	3	1	`0x1001D71`	— (write-through)	0
13	3	0	—	— (write-through)	—
14	3	0	—	— (write-through)	—
15	3	0	—	— (write-through)	—

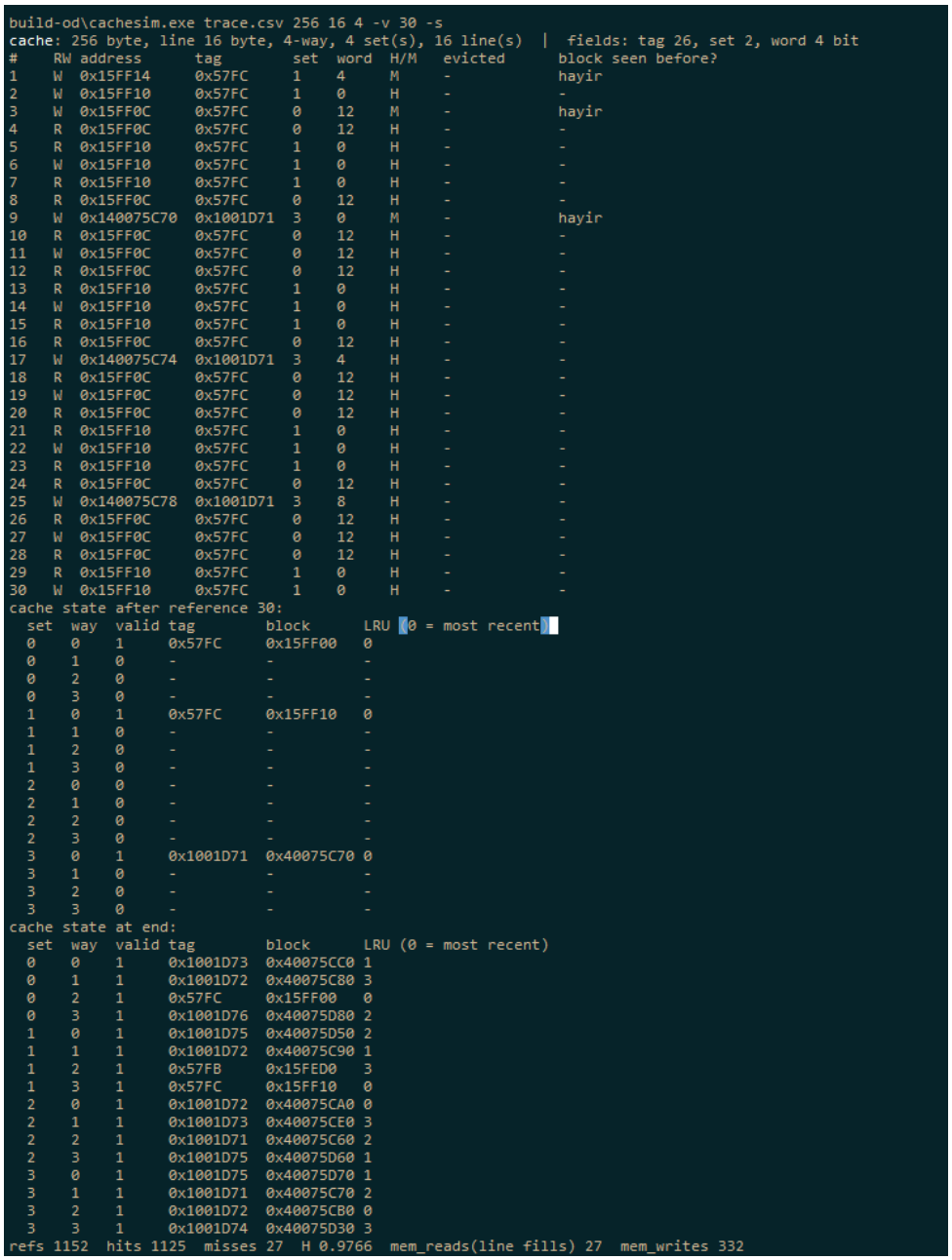
Write policy'nin bellek trafiğine etkisi: ilk 30 referanstaki 13 store'un her biri ana belleğe de yazılır (13 bellek yazması). Write-back olsaydı bu 13 yazma cache'te kalır, bellek trafiği yalnızca dirty line'lar yerinden çıkarıldığında oluşurdu; 30. referansta hiçbir line yerinden çıkmadığı için bu sayı 0 olurdu.

4d. Tam simülasyon sonuçları:

Ölçü	Elle (ilk 30)	Araç (ilk 30)	Araç (tüm iz)
Referans	30	30	1152
Hit	27	27	1125
Miss	3	3	27
Hit ratio H	0,9000	0,9000	**0,9766**
Ana belleğe yazma sayısı	13	13	332

Kullanılan araç ve ayarları: projenin C99 simülatörü `tools/cachesim.c` (kaynak kodu depoda; CMake hedefi `cachesim`); çağrı: `cachesim trace.csv 256 16 4 -v 30 -s`. Write-through + write-allocate, LRU. Bütün çıktılar `listings/cachesim_report.txt` dosyasındadır. Tüm izdeki 27 miss'in 20'si compulsory (dokunulan 20 farklı blok), 7'si conflict/capacity miss'tir (doğrulama döngüsünde `v` satırlarının yeniden okunması ve `main`'in `ret`'i).

- ☒ Elle ve araçla yapılan simülasyon ilk 30 referansta birebir örtüşüyor.



```
build-od\cachesim.exe trace.csv 256 16 4 -v 30 -s
cache: 256 byte, line 16 byte, 4-way, 4 set(s), 16 line(s)  |  fields: tag 26, set 2, word 4 bit
#    RW address      tag        set  word  H/M   evicted      block seen before?
1    W  0x15FF14     0x57FC     1    4     M     -            hayir
2    W  0x15FF10     0x57FC     1    0     H     -            -
3    W  0x15FF0C     0x57FC     0    12    M     -            hayir
4    R  0x15FF0C     0x57FC     0    12    H     -            -
5    R  0x15FF10     0x57FC     1    0     H     -            -
6    W  0x15FF10     0x57FC     1    0     H     -            -
7    R  0x15FF10     0x57FC     1    0     H     -            -
8    R  0x15FF0C     0x57FC     0    12    H     -            -
9    W  0x140075C70  0x1001D71  3    0     M     -            hayir
10   R  0x15FF0C     0x57FC     0    12    H     -            -
11   W  0x15FF0C     0x57FC     0    12    H     -            -
12   R  0x15FF0C     0x57FC     0    12    H     -            -
13   R  0x15FF10     0x57FC     1    0     H     -            -
14   W  0x15FF10     0x57FC     1    0     H     -            -
15   R  0x15FF10     0x57FC     1    0     H     -            -
16   R  0x15FF0C     0x57FC     0    12    H     -            -
17   W  0x140075C74  0x1001D71  3    4     H     -            -
18   R  0x15FF0C     0x57FC     0    12    H     -            -
19   W  0x15FF0C     0x57FC     0    12    H     -            -
20   R  0x15FF0C     0x57FC     0    12    H     -            -
21   R  0x15FF10     0x57FC     1    0     H     -            -
22   W  0x15FF10     0x57FC     1    0     H     -            -
23   R  0x15FF10     0x57FC     1    0     H     -            -
24   R  0x15FF0C     0x57FC     0    12    H     -            -
25   W  0x140075C78  0x1001D71  3    8     H     -            -
26   R  0x15FF0C     0x57FC     0    12    H     -            -
27   W  0x15FF0C     0x57FC     0    12    H     -            -
28   R  0x15FF0C     0x57FC     0    12    H     -            -
29   R  0x15FF10     0x57FC     1    0     H     -            -
30   W  0x15FF10     0x57FC     1    0     H     -            -
cache state after reference 30:
  set  way  valid tag        block      LRU (0 = most recent)
  0    0    1     0x57FC     0x15FF00   0
  0    1    0     -          -          -
  0    2    0     -          -          -
  0    3    0     -          -          -
  1    0    1     0x57FC     0x15FF10   0
  1    1    0     -          -          -
  1    2    0     -          -          -
  1    3    0     -          -          -
  2    0    0     -          -          -
  2    1    0     -          -          -
  2    2    0     -          -          -
  2    3    0     -          -          -
  3    0    1     0x1001D71  0x40075C70 0
  3    1    0     -          -          -
  3    2    0     -          -          -
  3    3    0     -          -          -
cache state at end:
  set  way  valid tag        block      LRU (0 = most recent)
  0    0    1     0x1001D73  0x40075CC0 1
  0    1    1     0x1001D72  0x40075C80 3
  0    2    1     0x57FC     0x15FF00   0
  0    3    1     0x1001D76  0x40075D80 2
  1    0    1     0x1001D75  0x40075D50 2
  1    1    1     0x1001D72  0x40075C90 1
  1    2    1     0x57FB     0x15FED0   3
  1    3    1     0x57FC     0x15FF10   0
  2    0    1     0x1001D72  0x40075CA0 0
  2    1    1     0x1001D73  0x40075CE0 3
  2    2    1     0x1001D71  0x40075C60 2
  2    3    1     0x1001D75  0x40075D60 1
  3    0    1     0x1001D75  0x40075D70 1
  3    1    1     0x1001D71  0x40075C70 2
  3    2    1     0x1001D72  0x40075CB0 0
  3    3    1     0x1001D74  0x40075D30 3
refs 1152  hits 1125  misses 27  H 0.9766  mem_reads(line fills) 27  mem_writes 332
```

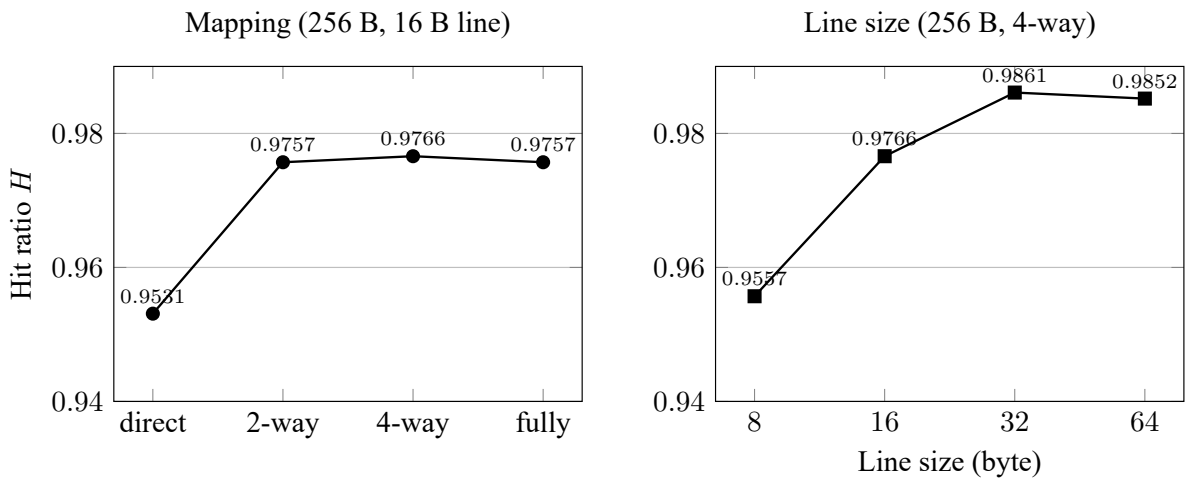
Simülatör çıktısı (`ss_cache.png`): ilk 30 referans, 30. referanstan sonraki ve programın sonundaki cache durumu, tüm izin toplamları.

4e. Tasarım uzayı (cache boyu sabit = 256 byte):

Mapping (line boyu sabit = 16 B)	Hit	Miss	H
Direct	1098	54	0,9531
2-way set-associative	1124	28	0,9757
4-way set-associative	1125	27	**0,9766**
Fully associative (LRU)	1124	28	0,9757

Line boyu (mapping sabit = 4-way)	Hit	Miss	H

8 B	1101	51	0,9557
16 B	1125	27	0,9766
32 B	1136	16	**0,9861**
64 B	1135	17	0,9852



Grafik dosyası ve eğilimlerin locality ile açıklanması (grafik: `figures/cache_sweep.pdf`): *Mapping.* Direct-mapped en kötüsüdür (54 miss): 16 line'lık direct-mapped cache'te aralarında tam 256 byte'ın katı olan iki blok aynı line'a düşer. `rle_encode` ve `rle_decode`'un stack frame'i (`0x15FEC0`, `0x15FED0`) `v`'nin sonu ve `enc`'in başıyla (`0x40075CC0`, `0x40075CD0`) aynı line'lara düşer; döngüler her iterasyonda hem yerel değişkene hem diziye eriştiği için iki blok birbirini sürekli çıkarır (thrashing). Doğrulama döngüsünde de `v[i]` ile 256 byte ötedeki `dec[i+16]` (`0x40075C70` ve `0x40075D70`) çatışır. 2-way bu çatışmayı büyük ölçüde çözer: aynı set'teki iki blok birlikte durabilir. Fully associative, 4-way'den biraz *kötüdür*: program 20 farklı bloğa dokunur ama cache 16 line tutar; döngüsel yeniden erişimde LRU tam da birazdan gerekecek line'ı çıkarır. Yani kalan miss'ler conflict değil capacity miss'tir. *Line boyu.* 8'den 32 byte'a büyüdükçe H artar: diziler sıralı taranır, bir miss komşu elemanları da getirir (spatial locality), compulsory miss sayısı düşer. 64 byte'ta H biraz düşer: cache boyu sabit olduğundan yalnızca 4 line kalır (4-way = tek set); stack, `v`, `dec` ve globaller aynı dört line için yarışır ve temporal locality kaybedilir. Gerçek L1 cache'lerin 64 byte line kullanması, onların 48 KB gibi çok daha büyük olmasıyla tutarlıdır.

4f. Ortalama erişim süresi: $T_s = H \times T_1 + (1 - H) \times (T_1 + T_2)$, $T_1 = 1$, $T_2 = 25$ cycle (varsayım).

Durum	H	T_s (cycle)	Speedup (cache'siz duruma göre)
Cache yok	0	$T_2 = 25$	1
Kişiye özel cache	0,9766	$0{,}9766 \times 1 + 0{,}0234 \times 26 =$ **1,586**	$25/1{,}586 =$ **15,76**

4g. Kod değişikliği: `main_v2.c`, `main.c`'den tek bir değişiklikle ayrılır: `dec`'in eleman tipi 8 bit olur (bunun için diğer tiplerle aynı biçimde bir `u8` typedef'i eklenir). Makine kodu ve veri erişimleri, `typedef` satırının kaydırdığı C satır numaraları dışında aynıdır.

	Önce	Sonra

Değiştirilen C satır(lar)ı	s. 17: `s32 dec[N];`	s. 19: `u8 dec[N];` (s. 13: `typedef unsigned char u8;`)
Y7 çıktısı (aynı olmalı)	11, 45, 1	11, 45, 1 ✓
H	0,9766 (27 miss)	**0,9835** (19 miss)
T_s (cycle)	1,586	**1,412**

Değişikliğin gerekçesi. Miss analizi, 27 miss'in 7'sinin conflict/capacity olduğunu gösterdi: program 20 farklı bloğa dokunuyor, cache 16 line tutuyor ve doğrulama döngüsü `v`'yi yeniden okuduğunda `dec` satırları onu çıkarmış oluyor. `dec` yalnızca 0 ve 1 tutar; byte olarak saklanınca 96 byte'tan 24 byte'a, 6 line'dan 2 line'a iner. Bu hem 4 compulsory miss'i (`dec`'in kalkan 4 line'ı) hem de 4 conflict miss'i siler: toplam ayak izi 16 line'a düşer ve doğrulama döngüsü `v`'yi cache'te bulur. Önce iki başka değişiklik de denendi ve her biri yalnızca 1 miss kazandırdı: doğrulama döngüsünü tersten yürütmek (LRU'nun tuttuğu dizi sonlarından başlamak) ve `dec`'i 1–4 line kaydırmak (array padding). İkisinin de sınırlı kalması, sorunun yanlış set eşlemesi değil kapasite olduğunu gösterdi; bu yüzden ayak izini küçülten değişiklik seçildi. Bir yan etki: byte karşılaştırması `movzx` gerektirdiği için `main_v2` 24 instruction daha fazla yürütür (2112), ama veri referansı sayısı aynıdır (1152).

Adım 5 — Performans ve architecture/organization (final paketiyle: 16 Kasım, Pazartesi)

5a-1. Dinamik instruction dağılımı ve CPI (CPI_i değerleri yönergedeki model varsayımıdır). Sınıflama kuralı: `j*`, `call`, `ret` → branch; `imul`, `idiv` → çarpma (modelde bölme sınıfı yoktur, çarpmaya sayılmıştır); belleğe yazan → store; bellekten okuyan → load; geri kalan → ALU. Kaynak: `listings/exec_O0.csv`, `exec_O1.csv`; hesap `tools/perf.py`.

Sınıf	CPI_i (varsayım)	I_i (-O0)	I_i (-O1)
ALU	1	601	601
Çarpma	3	26	26
Load	1	792	99
Store	1	330	68
Branch / jump	2	339	194
Ic (toplam)		**2088**	**988**
Ortalama CPI		**1,1873**	**1,2490**

5a-2. Süre ve MIPS rate (f = **1,5** GHz, τ = **0,667** ns; varsayım):

Ölçü	Formül	-O0	-O1
T (bellek etkisi yok)	Ic × CPI × τ	1652,7 ns	822,7 ns
MIPS rate	$f/(\text{CPI} \times 10^6)$	1263,4	1201,0
m (instruction başına data referansı)	data ref. / Ic	$1152/2088 = 0{,}5517$	$173/988 = 0{,}1751$
T, cache yok ($k = T_2$)	Ic × [p + (m × k)] × τ	20 852,7 ns	3 706,0 ns
T, cache var ($k = T_s$)	Ic × [p + (m × k)] × τ	2 870,7 ns ($T_s = 1{,}586$)	1 321,3 ns ($T_s = 4{,}324$)

$p =$ ortalama CPI; instruction fetch'lerin 1 cycle'da karşılandığı varsayılmıştır (varsayım). -O1'in T_s'i aynı cache konfigürasyonunda `trace_O1.csv` ile simüle edilmiştir (H = 0,8671).

Karşılaştırma	Değer
Speedup -O0 → -O1 (Eq. 2.1, cache var)	$2870{,}7/1321{,}3 =$ **2,17**
Speedup cache yok → cache var (-O0)	$20852{,}7/2870{,}7 =$ **7,26**

MIPS rate'in iki derlemeyi karşılaştırmada neden yanıltıcı olabileceği, kendi sayılarımla: MIPS'e göre -O0 (1263) -O1'den (1201) %5 daha "hızlı" görünür, oysa -O0 aynı işi 2,11 kat daha fazla instruction'la yapar (2088'e karşı 988) ve gerçekte 2,17 kat yavaştır (2870,7 ns'ye karşı 1321,3 ns). -O0'ın fazladan instruction'ları çoğunlukla CPI'si 1 olan spill load/store'larıdır; bunlar ortalama CPI'yi düşürüp MIPS'i yükseltir ama işe katkı yapmaz. MIPS instruction sayar, iş saymaz; farklı instruction dizileri üreten iki derlemeyi yalnızca toplam süre (T) adil karşılaştırır. Ayrıca -O0'ın yüksek H'si (0,9766 / 0,8671) da büyük ölçüde stack trafiğinin sürekli hit olmasından gelir.

5a-3. Amdahl yasası ($N_c =$ **6**; varsayım):

Alan	Değer
Paralelleştirilebilen kısım (C satırları)	s. 64–66: doğrulama döngüsü (`for ... if (dec[i] != v[i]) decode_ok = 0;`)
Bağımsızlık gerekçesi (veri bağımlılığı)	i'nci iterasyon yalnızca `dec[i]` ve `v[i]`'yi okur; hiçbir iterasyon başka bir iterasyonun yazdığı değeri okumaz. Tek yazma `decode_ok = 0`'dır ve hep aynı sabiti yazdığı için sıra önemsizdir (bir AND indirgemesi olarak da yazılabilir). Diğer döngüler seridir: LCG'de x_{i+1}, x_i'ye bağlıdır; `rle_encode`'da `i += run` sonraki iterasyonun başlangıcını belirler; `rle_decode`'da `o` önceki bütün koşuların toplamıdır.
f (yürütme süresindeki pay)	0,1639 (s. 64–68 instruction'larının cache'li T içindeki payı: $\sum(\text{CPI}_i + m_i \times T_s)$ / toplam; -O0)
Speedup $= 1/[(1-f) + f/N_c]$	$1/(0{,}8361 + 0{,}1639/6) =$ **1,158**
$N_c \to \infty$ limiti	$1/(1-f) =$ **1,196**

5b. Architecture ve organization (en az 10 özellik). Stallings'in tanımı (Ch. 1.1): architecture, programcıya görünen ve programın mantıksal yürütülmesini doğrudan etkileyen özelliklerdir; organization, bu özellikleri gerçekleştiren ve programcıya şeffaf olan donanım ayrıntılarıdır.

#	Özellik	Architecture / Organization	Gerekçe (Stallings tanımı)	Değişseydi Y7 değişir miydi? Kanıt
1	Instruction set ve encoding (x86-64, 1–15 byte)	Architecture	Derleyici ve programcı bunu görür, programın anlamını belirler	Yeniden derlenmeden çalışmaz; derlenince aynı: -O0 ve -O1 farklı instruction dizileri, Y7 aynı (2b)
2	Genel amaçlı register sayısı (16)	Architecture	Programcıya görünür	Hayır (derlenince): -O0 4, -O1 7 register kullanır, Y7 aynı
3	Adresleme modları (base + index×scale + disp, RIP-relative)	Architecture	Instruction anlamının parçası	Hayır: -O1 farklı modlar (`[rax+rcx]`) kullanır, Y7 aynı

#	Özellik	Architecture / Organization	Gerekçe (Stallings tanımı)	Değişseydi Y7 değişir miydi? Kanıt
4	Tamsayı genişliği (`int` 32 bit, mod 2^{32} aritmetik)	Architecture	Veri tiplerinin bit sayısı (Stallings'in örneği)	**Evet**: LCG mod 2^{32} taşmaya dayanır. 16 bit aritmetikte `x >> 16` hep 0 olur, `v` tamamen 0 çıkar ve Y7 = (1, 4, 1) olur
5	Interrupt mekanizması (IDT, vektör, stack frame)	Architecture	Sistem programcısına görünür	Hayır: interrupt programa şeffaftır, bağlam aynen geri yüklenir (3c)
6	Byte sırası (little-endian)	Architecture	Bellekteki veri biçimi programcıya görünür	Hayır: program hiçbir değişkeni byte byte yorumlamaz; yalnızca makine kodu düzeni değişir (2c)
7	Cache boyu (256 B)	Organization	Programcıya şeffaf donanım	Hayır: aşağıdaki tabloda 7 konfigürasyonda iz aynı, yalnızca H ve süre değişir
8	Mapping (4-way set-assoc.)	Organization	Programcıya şeffaf	Hayır (4e)
9	Line boyu (16 B)	Organization	Programcıya şeffaf	Hayır (4e)
10	Write policy (write-through)	Organization	Programcıya şeffaf	Hayır: yalnızca ana belleğe yazma sayısı değişir (332)
11	Data bus genişliği (64 bit)	Organization	Aktarım genişliği bir gerçekleştirme ayrıntısıdır	Hayır: yalnızca bus cycle sayısı değişir ($26 \rightarrow 58$, 3d)
12	Saat frekansı (1,5 GHz)	Organization	Programcıya şeffaf	Hayır: yalnızca $T = \mathrm{Ic} \times \mathrm{CPI}/f$ değişir
13	Çarpma donanımı (`imul`'un gerçekleştirilişi, CPI 3)	Organization	Stallings: çarpma instruction'ının varlığı architecture, nasıl yapıldığı organization	Hayır: yalnızca cycle sayısı değişir
14	Fiziksel adres genişliği (bu CPU'da 39 bit)	Organization	Aynı ISA (64 bit adres) farklı gerçekleştirmeler	Hayır: programın adresleri 33 bite sığar

Son sütunun testi: Aşama 4'teki bütün cache konfigürasyonları aynı `trace.csv`'yi işler; cache değerleri değil yalnızca erişim süresini değiştirir, Y7 her birinde (11, 45, 1) kalır. Süre ise değişir (T, -O0, cache var):

Konfigürasyon (256 B)	H	T_s (cycle)	T (ns)	Y7
direct, 16 B	0,9531	2,172	3320,7	11, 45, 1
2-way, 16 B	0,9757	1,608	2887,4	11, 45, 1
4-way, 16 B (kişiye özel)	0,9766	1,586	2870,7	11, 45, 1
fully, 16 B	0,9757	1,608	2887,4	11, 45, 1
4-way, 8 B	0,9557	2,107	3270,7	11, 45, 1
4-way, 32 B	0,9861	1,347	2687,4	11, 45, 1
4-way, 64 B	0,9852	1,369	2704,1	11, 45, 1

Ch. 1.2'deki dört temel fonksiyon:

Fonksiyon	Kernel'den örnek instruction (adres, assembly)	Açıklama
Data processing	`0x140001209 idiv ecx`	ALU veriyi işler: $1100 \div 24 = 45$
Data storage	`0x14000120B mov [rip+0x74A53], eax`	Sonuç (`ratio_x100`) belleğe yazılır ve program bittikten sonra orada kalır (Y7)
Data movement	`0x140001168 mov eax, [rsp+0x10]`	Veri bellekten register'a taşınır. Program dış dünyayla I/O yapmaz; Stallings'in anlamındaki dışarıya taşıma, sonuçların bellekten debugger ile okunmasıyla olur
Control	`0x140001170 ret` ve `0x140001231 jge`	Instruction akışını yönetir: dönüş adresine sıçrama, koşullu dallanma

Final teslim (16 Kasım Pazartesi; sözlü savunma 20 Kasım Cuma)

Teslim kontrol listesi:

- ☒ Bu tutanak, tüm adımları dolu olarak (PDF: `CMPE3209_Tutanak.pdf`, düzenlenebilir sürüm: `CMPE3209_Tutanak.tex`)
- ☒ C kaynak kodu (`main.c`) ve sonraki 4g sürümü (`main_v2.c`)
- ☒ `asm_O0.s`, `asm_O1.s` ve objdump -d çıktısı (`listings/`)
- ☒ `trace.csv` ve cache simülasyon çıktıları (`listings/cachesim_report.txt`; simülatör kaynağı `tools/cachesim.c`)
- ☒ Ekran görüntüleri: machine code doğrulaması (`ss_m2_disasm.png`), program çıktıları (`ss_globals.png`), cache durumu (`ss_cache.png`)
- ☒ 4e grafiği (`figures/cache_sweep.pdf`)
- ☐ Git deposunda M1, M2 ve final etiketleri

Kilometre taşı kaydı (öğretim üyesi doldurur):

Kilometre taşı	Son tarih	Teslim tarihi	Git etiketi	Paraf
M1 Öneri (Adım 0–1)	2 Kasım (Pzt)			
M2 Assembly ve machine code (Adım 2)	9 Kasım (Pzt)			
M3 CPU, interrupt, bus (Adım 3)	16 Kasım (Pzt, final paketiyle)			
M4 Cache ve performans (Adım 4–5)	16 Kasım (Pzt, final paketiyle)			
Final ve sözlü savunma	Teslim 16 Kasım (Pzt); savunma 20 Kasım (Cuma)			

YZ kullanım beyanı. Kullanılmadıysa ilk satıra "Kullanılmadı" yazın. Beyan edilmemiş kullanım akademik dürüstlük ihlalidir.

Araç ve sürümü	Adım	Ne için kullanıldı	Örnek prompt	Çıktıyı nasıl doğruladım

Claude Code (Anthropic, Claude Opus)	0–5	Rapor yazımında yardımcı: tutanağın Türkçe metni, tablo ve LaTeX dizgisi; Intel SDM Vol. 3A'daki interrupt mekanizması için bölüm ve sayfa referanslarının bulunması (3c)	"Tutanağı resmi forma benzer biçimde doldur"; "Intel SDM'de 64-bit mode interrupt stack frame bölümünü ve sayfasını bul"	Metindeki her sayıyı kendi derleme, debugger ve simülasyon çıktılarımla karşılaştırdım; SDM referanslarını 253668-080US PDF'inin içindekiler tablosundan kontrol ettim
Claude Code (Anthropic, Claude Opus)	2, 4, 5	Yardımcı araçların yazılması: `tools/mktrace.py` (debugger kaydından `trace.csv`), `tools/c2asm.py` (C satırı → adres eşlemesi), `tools/perf.py` (5a hesapları), `cmake/listings.cmake` (listeleme)	"Debugger'ın tek adım kaydından her load ve store için etkin adresi hesaplayıp trace.csv yazan bir Python scripti yaz"	İz M1'deki ölçümle birebir tutar (1152 referans, 961 stack); iki ayrı çalıştırma aynı izi verir; simülatör ilk 30 referansta elle simülasyonla örtüşür; makine kodu objdump ve Visual Studio ile aynıdır. Cache simulatörü şahsım tarafından yazılmıştır.

Öğrenci beyanı. Bu tutanaktaki C kodunu, elle machine code kodlamasını, instruction cycle izleme tablosunu ve elle cache simülasyonunu kendim yaptım. Başka bir öğrencinin çalışmasını kopyalamadım ve kendi çalışmamı paylaşmadım. YZ kullanımımı eksiksiz beyan ettim. Sözlü savunmada bu çalışmayı açıklayamamamın puanımı etkileyeceğini biliyorum.

Ad Soyad	Öğrenci numarası	Tarih	İmza
Mehmet Batuhan Özer	5001230025		

Değerlendirme (öğretim üyesi doldurur)

Bileşen	Azami	Puan	Not
A1 Öneri ve parametre hesabı (Adım 0–1)	5		
A2 Assembly ve machine code (Adım 2)	15		
A3 CPU, interrupt, bus (Adım 3)	25		
A4 Cache (Adım 4)	25		
A5 Performans ve architecture/organization (Adım 5)	10		
Tutanak ve rapor kalitesi	5		
Sözlü savunma	15		
Geç teslim kesintisi	—		
Toplam	**100**		

Sözlü savunma kaydı:

Madde	Sorulan	Sonuç (başarılı / kısmen / başarısız)	Puan
1. Canlı encode (-O1 listesinden)			
2. Adres alanları ve hit/miss			
3. Interrupt k+1'de gelseydi			
4. Tasarım kararı savunması			

☐ 1. ve 2. maddelerin ikisi de başarısız: A2–A4 puanları 0.5 ile çarpıldı.

Öğretim üyesi	Tarih	İmza